UNIVERSIDAD POPULAR DEL CESAR
Ingeniería de Sistemas e Informática

Xavi Acosta Maldonado

GUÍA DEL TALLER

Refactorización a Código Limpio

Sistema de Facturación — Restaurante El Buen Sabor

Programación de Computadores III · SS462 · Cuarto Semestre
Docente: Ing. Alfredo David Bautista Romero

1. ¿Qué van a encontrar?

El proyecto RestauranteBuenSabor es un sistema de facturación para un restaurante que funciona correctamente, pero está escrito con el máximo de malas prácticas posibles. Su misión es refactorizarlo hasta convertirlo en código limpio, sin alterar el comportamiento del programa.

> ⚠ *El programa produce resultados correctos. El problema no es que falle — el problema es que es imposible de mantener, entender y modificar. Eso es exactamente lo que pasa en la industria real.*

1.1 Estructura de archivos entregada

```
RestauranteBuenSabor/
├── src/
│   └── restaurante/
│       ├── Main.java        — punto de entrada y menú interactivo
│       ├── Datos.java       — almacén global de datos del sistema
│       ├── Proceso.java     — lógica de cálculo de la factura
│       ├── Utilidades.java  — métodos auxiliares
│       └── Imprimir.java    — todo lo que se imprime en pantalla
└── README.md                — instrucciones para compilar y ejecutar
```

2. ¿Qué hace el sistema?

El programa simula la operación de caja de un restaurante. Al ejecutarlo aparece un menú con 6 opciones que se repite en bucle hasta que el usuario elige salir:

```
========================================
    RESTAURANTE EL BUEN SABOR
    Calle 15 #8-32, Valledupar
    NIT: 900.123.456-7
========================================
1. Ver carta
2. Agregar producto al pedido
3. Ver pedido actual
4. Generar factura
5. Nueva mesa
0. Salir
========================================
Seleccione una opcion:
```

2.1 Carta del restaurante (Opción 1)

#	Producto	Precio
1	Bandeja Paisa	$32.000
2	Sancocho de Gallina	$28.000
3	Arepa con Huevo	$8.000

4	Jugo Natural	$7.000
5	Gaseosa	$4.500
6	Cerveza Poker	$6.000
7	Agua Panela	$3.500
8	Arroz con Pollo	$25.000

2.2 Reglas de negocio (Opción 4 — Generar factura)

- Subtotal: suma de precio × cantidad para cada producto pedido.
- Descuento 5%: se aplica si hay más de 3 productos diferentes en el pedido.
- IVA 19%: se calcula sobre el subtotal después del descuento.
- Propina 10%: se aplica si el subtotal (antes del IVA) supera $50.000.
- Total: subtotal con descuento + IVA + propina.

Ejemplo de factura generada (Bandeja Paisa x2 + Jugo Natural x2):

```
FACTURA No. 001
----------------------------------------
Bandeja Paisa        x2      $64.000
Jugo Natural         x2      $14.000
----------------------------------------
Subtotal:                   $78.000
IVA (19%):                  $14.820
Propina (10%):              $ 9.282
----------------------------------------
TOTAL:                      $102.102
```

3. Catálogo de Malas Prácticas

A continuación, se documenta cada mala práctica presente en cada archivo, con su ubicación y descripción. Usen esto como lista de verificación durante la refactorización.

3.1 Datos.java — El God Object

Almacena todos los datos del sistema en variables estáticas públicas. Cualquier clase puede leer y modificar cualquier dato directamente, sin control ni encapsulamiento.

#	Problema	Dónde	Descripción
1	Arrays paralelos	Línea 12, 13, 14 En nom[], p[], cant[]	Todos los arrays se pueden convertir en uno solo, ya que representan un mismo objeto
2	Todo public static	Línea 12 a la 18	Son clases publicas, y se pueden modificar, se recomienda hacerlas privadas y finales
3	Nombre hardcodeado	Variable x, línea 17	La variable x no indica que contiene
4	Variable tmp sin propósito	Línea 18	No explica que hace, se intuye que es para la parte vista
5	Sin constructor ni métodos	Todo el codigo	La clase solo contiene variables, no hay orden
6	Nombres de 1-3 letras	nom, p, ms, est, tot, nf, x, tmp	Son nombre que no indican nada que representan en el codigo

3.2 Proceso.java — El método que hace todo

Contiene la lógica de cálculo del total de la factura en un único método de más de 80 líneas.

#	Problema	Dónde	Descripción
7	hacerTodo() con múltiples responsabilidades	De la línea 12 a la 46	Un solo método hace muchísimas cosas, hay que crear un método por accion
8	IVA duplicado 3 veces	Línea 24, 38, 43	iva = aux * 0.19 aparece en cada if en vez de convertirlo en una variable o método.
9	Magic numbers sin nombre	0.19, 0.10, 0.05, 50000, 3	Porque son exactamente esas cantidades?
10	Condicionales anidados 4 niveles	De la línea 11 a la 35	if(cont>3) { if(sub>0) { if(aux>50000) { ... } else { ... } } } else { if(sub>50000) {...} }
11	Código comentado 'versión anterior'	De la línea 32 a la 35	Sub = sub*1,19, este codigo no se borró
12	procesar() con 7 parámetros	Línea 47	Duplica la lógica del método hacertodo
13	Variable aux innecesaria	Línea 22	El valor se puede sacar sin usar la variable
14	Comentarios que repiten el código	Lineas 25, 29, 39	dicen lo mismo para el mismo codigo
15	Comentarios de cierre inútiles	// fin while y fin if	Es obvio que al cerrar el corchete en esa acción, es porque se da fin a la condición

3.3 Utilidades.java — Nombres engañosos

Contiene métodos auxiliares: calcular, validar y reiniciar el pedido.

#	Problema	Dónde	Descripción
16	calcular() con 7 parámetros	Línea 12	Duplica lógica de proceso
17	validar() con efecto secundario oculto	Línea 29	Se llama validar y modifica a datos.tot y datos.tmp
18	Nombre del restaurante hardcodeado (3ra vez)	Lina 20	Según el nombre del método, debería validarse las cosas, no mostrar el nombre del restaurante
19	Bloque de ~20 líneas de código comentado	Línea 32 a la 47	Se intuye que era codigo antiguo, si no explica nada debería borrarse
20	Variables tmp y aux2 redundantes	Linea 13 y 21	Lo que hacen se pueden hacer sin ellas, solo copian valores

3.4 Imprimir.java — Código copiado y pegado

Contiene todos los métodos de impresión en consola.

#	Problema	Dónde	Descripción
21	imprimirFacturaCompleta() con 3 responsabilidades	De la linea 34 a la 78	Duplica procesos, según su nombre solo imprime, pero actualiza el estado de la mesa, lo cual debería estar fuera del metodo
22	Cálculo del total duplicado	De la linea 35 a la 50	Se repite en imprimirfacturacompleta y en imprimirfacturaresumen
23	80% del código idéntico entre métodos	De la línea 34 a la 90	Los mismo en ambos métodos, solo que el resumen contiene menos cosas

24	Nombre del restaurante hardcodeado (4ta vez)	Línea 53 a la 55 y de la 97 a la 99	Restaurante el buen sabor, la dirección y el nit, aparecen en ambos metodos
25	Formato printf repetido sin constante	Lineas 19, 28, 62, 104	"%-20s x%-6d $%,.0f%n" repetido sin extraerlo como constante nombrada.

3.5 Main.java — El caos central

Punto de entrada del programa. El método main mezcla presentación, validación y lógica de negocio en más de 100 líneas.

#	Problema	Dónde	Descripción
26	Método main de 100+ líneas	Main()	Todo esta en un solo metodo
27	Dos Scanner para el mismo System.in	Main()	Hay 2 scanner, sc y sc2, la idea es usar el mismo
28	Variables sin uso	boolean continuar	Nunca se usa
29	Lógica de negocio en el main	If y else aninados	Validaciones y actualizaciones mezclados con el menu
30	Nombre del restaurante hardcodeado (5ta vez)	Línea 18	Se muestra con el print, en vez de usar variables

3.6 Problemas de formato visual (todos los archivos)

#	Problema	Dónde	Descripción
31	Sin indentación	Todo el programa	Todo el codigo junto, no tiene formato
32	Llaves inconsistentes	Todo el programa	No hay algún patron con respeto a las llaves
33	Llaves de cierre apiladas	Main y proceso	Muchas llaves juntas, no se sabe a que pertenece
34	Múltiples statements por línea	Todo el programa	int a=0;boolean b=true;int c=0; en la misma línea separados por punto y coma.

Total: 34 malas prácticas identificadas y documentadas en el proyecto.

Referencia principal: Robert C. Martin — Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, 2009.

Fuente complementaria: Shahan Chowdhury — The Clean Code Handbook. FreeCodeCamp, Enero 2025.